

Verifier-Guided Repair of LLM-Generated Vendor CLI Configurations: A Controlled Fault-Injection Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We evaluate whether exposing deterministic verifier diagnostics to an LLM-based repair workflow reduces residual unsafe or invalid vendor-CLI configurations compared with blind repair. In a preregistered controlled-challenge paired design (vCLI_fault_injection_repair_2026_A_prereg_v1) executed under shared controlled-fault pairing semantics, we planned 40 confirmatory pairs and observed 39 complete pairs (one source generation invalid and excluded per the preregistered rule). Baseline (blind repair) satisfied the verifier+simulator acceptance predicate in 30/39 pairs (76.92%); guarded (verifier-guided) satisfied it in 38/39 pairs (97.44%). Paired counts: n11 = 29, n10 = 9 (guarded-only), n01 = 1 (baseline-only), n00 = 0. The paired absolute difference (guarded - baseline) = 0.20512820512820507 (20.51 percentage points); two-sided exact paired test $p = 0.021484375$; 95% paired-bootstrap percentile CI = [0.07692307692307693, 0.358974358974359] (bootstrap resamples = 10000, seed = 1729). Confirmatory execution used a declared model version gpt-5-mini and consumed 118 hosted-model calls (budget 120). A preregistered confirmatory harmful-overrepair aggregation was not present in the archived confirmatory summary and therefore remains unresolved for the confirmatory claim; the per-episode validation traces and the frozen mapping needed to compute that aggregation were recorded during execution and can be deterministically reaggregated by independent auditors. We present the preregistered primary result, clarify execution semantics, and state limitations and reproducibility guidance without altering the preregistered scientific record.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed as aides in network operations (NetOps) for tasks such as intent-to-configuration translation and automated remediation. A practical safety architecture is generate–verify–repair (GVR): candidate configurations are checked by deterministic verifiers and, when violations are found, verifier diagnostics are fed back to a generator/repair model to produce corrected candidates. Empirical work in code and Infrastructure-as-Code (IaC) settings reports that inserting deterministic verifiers can increase verifier-detected pass rates while exposing new failure modes (e.g., verifier-exploitation) and incurring interaction costs [1]–[4]. Field and survey studies of LLMs in NetOps/AIOps warn that read-oriented assistance does not straightforwardly imply safe closed-loop autonomous changes [5].

Motivation and research question. Controlled paired evidence on verifier-guided repair applied to vendor command-line interface (CLI) templates with preregistered realistic fault operators is sparse. Verifier-guided repair can only affect out-

comes when initial candidates trigger actionable diagnostics, so we used deterministic controlled faults that guarantee activation in a paired design that reuses the same controlled-fault candidate across arms. Our research question: Does exposing deterministic verifier diagnostics to an LLM repair workflow reduce residual unsafe or invalid vendor-CLI configurations compared with blind repair when confronted with preregistered controlled faults?

II. RELATED WORK

GVR pipelines have been evaluated across code, IaC, and DSL settings. Controlled evaluations and benchmark studies show that deterministic verifiers interposed between generation and acceptance raise verifier-detected pass rates and that supplying verifier diagnostics to a repair model increases the chance a corrected artifact satisfies the verifier on recheck [1], [2]. Complementary analyses document caveats: single-verifier iterative refinement can produce pathological behaviors where optimization toward the verifier proxy degrades the underlying target property (verifier-exploitation) [3], and verifier-mediated workflows impose measurable interaction and compute overheads (a "verifier tax") that depend on horizon and design choices [4]. Survey and field evidence for NetOps indicate that LLMs help with read-oriented tasks but provide weaker guarantees for safe closed-loop configuration changes without careful verification and conservative orchestration [5]. These prior findings motivate a controlled experiment that uses deterministic verifiers, ensures intervention activation via controlled fault injection, and instruments both correctness outcomes and safety failure modes.

III. METHODOLOGY

Preregistration and estimand. The study was preregistered under the identifier vCLI_fault_injection_repair_2026_A_prereg_v1. The primary estimand is the paired_success_rate_difference_guarded_minus_baseline, where a binary success indicator requires both the deterministic vendor-CLI verifier and a simulator-based compliance agreement.

Adapter and pairing semantics. Confirmatory execution used the hosted_netops_validator_feedback_repair_v2 adapter under its shared_controlled_fault_candidate semantics. Under

those semantics each task index received exactly one deterministically injected controlled-fault candidate which was reused unchanged across both experimental conditions; both conditions therefore received the same initial candidate. Exactly one sampled initial generation per task pair was performed and reused by both arms. Baseline denotes blind repair (repair without exposure of verifier diagnostics) and guarded denotes repair with deterministic validator diagnostics exposed to the repair model. For the executed adapter family and pairing semantics, both arms receive the same shared initial candidate and each arm receives its permitted repair opportunity; observed differences therefore isolate the effect of exposing diagnostics under a matched repair budget and matched source generation.

Model sampling and runtime parameters. The declared model used for confirmatory execution was gpt-5-mini. The archived model configuration records temperature = null (unset); null/unset is not evidence of deterministic sampling and does not imply temperature = 0. Exactly one sampled initial generation per pair was reused by both arms under the adapter contract; this design does not measure effects of independent first-pass sampling across conditions.

Controlled-challenge fault bank and scoring rules. We used a deterministic controlled-fault generator that injected realistic actionable violations (e.g., dropped required restore, protected-interface changes) into 40 preregistered task indices. The primary success predicate required agreement between a deterministic vendor-CLI validator and a simulator-based compliance check. The preregistration also froze a harmful-change labeling mapping that operationally marks an episode as harmful if post-repair validator traces include `PROTECTED_INTERFACE_CHANGE` or `UNINTENDED_STATE_CHANGE`; that frozen mapping and the per-episode validation traces were recorded during execution and are sufficient to deterministically reaggregate the preregistered harmful-overrepair statistic.

Statistical analysis plan. The predefined primary test was a two-sided exact paired binary test (exact McNemar or equivalent). We report discordant-pair counts (n_{10} , n_{01}), the absolute paired risk difference with a 95% paired-bootstrap percentile confidence interval (bootstrap resamples = 10000, seed = 1729), and the two-sided exact p-value. Missingness handling followed the preregistered rule 'complete_pair_primary' (exclude a pair only if both executions are unscorable); sensitivity analyses treating missing episodes as failures were also preregistered. All per-episode traces and scored artifacts required to recompute aggregations were recorded during execution.

IV. RESULTS

Execution accounting and primary confirmatory outcomes. Planned confirmatory pairs = 40; completed episodes correspond to 39 complete pairs (one source generation invalid and excluded under the preregistered rule). Hosted-model calls used = 118 ($\leq$ planned maximum 120). Source-generation attempts = 40 (source_valid_count = 39; source_invalid_or_failed_count = 1).

Primary confirmatory outcomes (complete_pairs = 39). Baseline (blind repair) success = 30/39 = 0.7692307692307693. Guarded (verifier-guided) success = 38/39 = 0.9743589743589743. Paired contingency counts (guarded, baseline): $n_{11} = 29$, $n_{10} = 9$ (guarded-only), $n_{01} = 1$ (baseline-only), $n_{00} = 0$. Paired estimate (guarded - baseline) = 0.20512820512820507 (20.51 percentage points). Exact two-sided paired test (McNemar exact): $p = 0.021484375$. 95% paired-bootstrap percentile CI = [0.07692307692307693, 0.358974358974359] (bootstrap_resamples = 10000, bootstrap_seed = 1729).

Missingness and excluded pair. The preregistered 'complete_pair_primary' rule excluded one pair because its source generation was invalid/unscorable, producing one both-missing pair; terminal error reasons for excluded episodes were recorded at execution time.

Preregistered harmful-overrepair confirmatory statistic: status. The preregistration included a confirmatory safety hypothesis bounding harmful over-repair. The archived confirmatory analysis produced the primary paired binary result but did not include the preregistered harmful-overrepair aggregation in its confirmatory summary. Because that aggregation is not present in the archived confirmatory summary, we do not present a confirmatory safety verdict here. The frozen harmful-change mapping and per-episode validator traces required to compute that preregistered statistic were recorded during execution and enable deterministic independent recomputation by auditors; any harmful-overrepair numbers computed outside the archived confirmatory summary are treated as exploratory.

V. DISCUSSION

Interpretation and mechanism. The confirmatory paired result shows a substantial increase in verifier+simulator-validated success when deterministic verifier diagnostics are exposed to the repair model under shared controlled-fault pairing semantics. The paired design isolates diagnostics exposure because both arms received the same initial controlled-fault candidate and identical repair budgets. Per-episode scoring traces show how diagnostics enabled targeted corrective edits: guarded episodes commonly contain validator_trace.violation_codes that identify protected fields or unintended-state changes, and exposing those codes appears to direct repairs away from protected-interface changes or to restore required command sequences that blind repair sometimes omitted.

Safety boundary and unresolved preregistered safety aggregation. The preregistered confirmatory harmful-overrepair aggregation remains unresolved in the archived confirmatory summary because that aggregation was not computed by the archived analysis executor. The frozen harmful-change mapping (an episode is harmful if post-repair validation contains `PROTECTED_INTERFACE_CHANGE` or `UNINTENDED_STATE_CHANGE`) and the per-episode validation traces were recorded during execution; independent auditors can deterministically recompute the preregistered safety statistic from those artifacts. Until such recomputation is shown

in a confirmatory artifact, any safety aggregation derived outside the archived confirmatory summary should be treated as exploratory.

Operational implications and verifier tax. Exposing diagnostics materially improved verifier-defined correctness on the tested controlled fault bank, but verifier-guided workflows incur additional model-verifier interactions: confirmatory execution consumed 118 hosted-model calls under the matched budget constraints. These counts allow exploratory computation of model-call cost per avoided failure, but assessing large-scale operational feasibility (the verifier tax tradeoffs) requires broader deployment-scale studies.

Verifier-exploitation, generality, and mitigations. Prior work documents verifier-exploitation risk when single-proxy verifiers are optimized in iterative loops [3]. Our short-horizon repair budget and shared-candidate pairing reduce exposure to long-horizon exploitation, but the risk persists in multi-step agentic pipelines. Mitigations include multi-verifier ensembles, explicit harmful-change constraints, calibrated abstention, and incorporating uncertainty quantification. The frozen harmful-change mapping used in this study is an example of an explicit constraint that can be enforced in follow-up experiments.

Threats to validity. Internal validity: model sampling non-determinism (temperature = null/unset) and exactly-one initial sampled generation per pair limit claims about deterministic generation effects; differences across arms reflect diagnostics exposure under the adapter contract. Construct validity: primary success required agreement of a deterministic verifier and a simulator; both components limit the operational meaning of "valid." External validity: confirmatory execution used a single declared model (gpt-5-mini) and a synthetic template-based controlled-fault bank, constraining generalization to diverse vendor equipment and live operator streams. Conclusion validity: complete_pairs = 39 provides reasonable precision for the observed primary contrast but limits power for per-fault-class inference; subgroup results are exploratory.

VI. LIMITATIONS

- Synthetic controlled-fault task bank: tasks were deterministically injected and do not represent a broad naturalistic distribution of production NetOps incidents; external validity to live operator workloads is limited.
- Single-model and single-adapter evaluation: confirmatory execution used only the declared model gpt-5-mini and one adapter family; results do not establish multi-model generality.
- Simulator-backed primary success predicate: primary success required agreement of deterministic verifier and simulator; simulator fidelity and verifier coverage constrain construct validity.
- Sample size and power: complete_pairs = 39 provides detectable effects of the observed magnitude but limits precision for subgroup and per-fault-class inference; exploratory subgroup analyses are not confirmatory.
- Operational cost/latency unresolved for deployment: execution was instrumented and counts were recorded, but

large-scale operational feasibility (verifier tax tradeoffs) requires larger-scale study.

- Verifier-exploitation and long-horizon agentic pathologies remain possible: this controlled setting mitigates some risks, but broader agentic workflows need additional defenses (multi-verifier designs, calibrated abstention, uncertainty quantification).
- Preregistered confirmatory harmful-overrepair test not present in archived confirmatory summary: the archived confirmatory analysis executor did not compute the preregistered harmful-overrepair aggregation; the per-episode traces and the frozen mapping required to compute it were recorded and are available for independent reaggregation, but the preregistered confirmatory safety verdict is therefore unresolved in this paper.

VII. CONCLUSION

In a preregistered controlled-challenge paired experiment using a hosted validator-feedback repair adapter and shared controlled-fault candidates, exposing deterministic verifier diagnostics to an LLM repair workflow produced a statistically detectable and substantial increase in verifier+simulator-validated configuration success (approx. +20.51 percentage points; $p = 0.021484375$; 95% CI [0.07692307692307693, 0.358974358974359]). This finding is bounded to the preregistered synthetic task bank, the declared model (gpt-5-mini), and the deterministic verifier and simulator used in execution. The preregistered confirmatory harmful-overrepair test was not present in the archived confirmatory summary and therefore remains unresolved for confirmatory inference; the frozen harmful-change mapping and the per-episode validation traces recorded during execution provide exact inputs for independent reconstruction of that preregistered safety statistic. We recommend multi-model, multi-verifier replications and larger-scale cost/latency studies to assess operational feasibility and safety at NetOps scale.

DISCLOSURE STATEMENT

Preregistration: `vCLI_fault_injection_repair_2026_A_prereg_v1`. Adapter family used for confirmatory execution: `hosted_netops_validator_feedback_repair_v2`. Declared model used for confirmatory execution: gpt-5-mini. Exact initial master prompt archived as `provenance/master_prompt.txt` (SHA-256: `f781dd7978328198146d586cf33e0f4b783c8db126bd0f16fcd13699455ebb10`). Human role and autonomy boundary: a Research Director selected the topic family and pre-lock constraints; after the autonomy lock the autonomous system performed literature verification, preregistration, experiment execution, analysis, and manuscript generation; no manual human editing of technical manuscript content occurred after the lock. Execution semantics: the archived model configuration records temperature = null (unset); null/unset is not evidence of deterministic sampling and does not imply temperature = 0. Pairing semantics: exactly one sampled initial generation per task pair was performed and reused by both arms under `shared_controlled_fault_candidate`

semantics; baseline denotes blind repair (no validator diagnostics exposed) and guarded denotes repair with deterministic validator diagnostics exposed. The preregistered harmful-change labeling mapping used to define the planned safety aggregation was frozen before execution; the mapping and per-episode validation traces required to compute the preregistered harmful-overrepair aggregation were recorded during execution. All per-episode traces, scored artifacts, and transformation mappings needed to reproduce the preregistered secondary aggregations were recorded in the archived evidence bundle. The study followed the preregistered stopping rule.

REFERENCES

- [1] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [2] <https://arxiv.org/abs/2607.20478>
- [3] Arnav Gupta, "Verifier Exploitation in NLI-Guided Iterative Refinement: A Controlled Empirical Analysis", 2026, 10.21203/rs.3.rs-10187492/v1
- [4] <http://arxiv.org/abs/2603.19328>
- [5] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729